PLATEFORME DE TRAITEMENT DES PAIEMENTS

ARCHITECTURE DU PIPELINE DE DONNEES



Nature du document	
Version	**1.1**
Client	**Étude de cas – STRIPE**
Date de la version	**15/10/2026**
Description de la version	**Bloc 2 - certification RNCP41993**
Modules concernés	**Livrable 4**
Auteur	**Ronan GUILLOU**

Table des matières

1 Concepts clés 3
1.1 Définitions 3
1.2 Principes 3
2 Vue d'ensemble 3
3 Vue détaillée par flux 4
3.1 De l'OLTP vers le lakehouse 4
3.1.1 Écriture du paiement 4
3.1.2 Publication dans Kafka 5
3.1.3 Arrivée dans Bronze 5
3.1.4 Transformation : Bronze, Silver, Gold 5
3.2 Du NoSQL vers le lakehouse 5
3.2.1 Collecte et topics 5
3.2.2 Stockage dans DocumentDB 5
3.2.3 Export de nuit vers Bronze 6
3.3 Alimentation de Redis et sorties de Gold 6
3.3.1 Redis 6
3.3.2 Autres sorties de Gold 6
3.4 Boucle de feedback : étiquettes de fraude 6
3.4.1 Collecte 6
3.4.2 Arrivée dans Bronze 6
3.4.3 Transformation 6
3.4.4 Réentraînement 6
4 Orchestration 7
5 Monitoring et reprise 7
5.1 Monitoring 7
5.2 Reprise sur incident 7

Note sur l'usage de l'intelligence artificielle

Pour rédiger ce document, j'ai utilisé un assistant d'IA générative comme outil d'aide à la rédaction. Il est intervenu sur quatre points : la correction de l'orthographe et de la syntaxe, la proposition de structures et de brouillons de texte que j'ai relus et modifiés, la comparaison d'idées et d'options avant décision, et la vérification de la cohérence d'ensemble.

Les choix retenus, la sélection des besoins et la validation finale de chaque chapitre relèvent de ma responsabilité.

1 Concepts clés

Ce document décrit comment les données circulent entre **les trois systèmes** de la plateforme (OLTP PostgreSQL, lakehouse Databricks, NoSQL DocumentDB) ainsi que vers le cache Redis et l'entraînement du modèle de détection de fraude : les flux, leur mode (temps réel ou par lots), leur orchestration et leur surveillance.

1.1 Définitions

Terme	Définition
Outbox	Table écrite dans la même transaction que le paiement, qui contient les événements à publier en streaming.
CDC	Change Data Capture : Technique qui lit les nouvelles lignes d'une base et les publie, sans ralentir les écritures.
Kafka, topic	Service de messagerie durable et rejouable. Un topic regroupe les messages d'un même type.
Au moins une fois	Garantie de livraison : Un message n'est jamais perdu, mais peut arriver en double.
Idempotence	Rejouer un traitement une ou dix fois donne le même résultat.
DLQ	Dead Letter Queue : File où l'on isole les messages impossibles à traiter, pour analyse.
Quarantaine	Table qui conserve les lignes rejetées à la validation, avec leur motif.
Médaillon	Trois couches de données : Bronze (brut), Silver (propre, sans doublon), Gold (prêt pour l'usage).
Streaming, batch	Streaming : traitement continu, en quelques secondes. Batch : traitement par lots planifiés.
TTL	Durée de vie d'une donnée avant suppression automatique.
Orchestration	Ordonnancement des étapes d'un pipeline, avec dépendances, relances et alertes.

1.2 Principes

- **Le paiement et son événement sont atomiques.** Le paiement, l'évaluation de fraude et la ligne d'outbox sont écrits dans la même transaction. Aucun événement n'est perdu ni fantôme, et si Kafka tombe, les paiements continuent.
- **Le temps réel est réservé à ce qui est critique.** Seuls l'écriture du paiement (moins de 100 ms) et les compteurs de fraude sont alimentés en continu. Les analyses BI, les features froides et l'entraînement se contentent de lots (heure, jour).
- **Tous les consommateurs sont idempotents.** Kafka livrant « au moins une fois », chaque traitement doit supporter un rejeu sans créer de doublon.
- **Bronze est la source de rejeu.** Il conserve les messages tels que reçus : si Silver ou Gold sont faux, on les recalcule depuis Bronze.
- **Aucune donnée rejetée n'est perdue.** Les lignes invalides vont en quarantaine ou en DLQ avec leur motif, pour correction à la source et inscription en log.
- **Deux flux d'entrée séparés**. Critique et secondaire. Le flux critique (paiement) passe par l'outbox, dans la même transaction que le paiement. Le flux secondaire (logs, clics, sessions, avis, et fraudes confirmées par la banque émettrice).

2 Vue d'ensemble

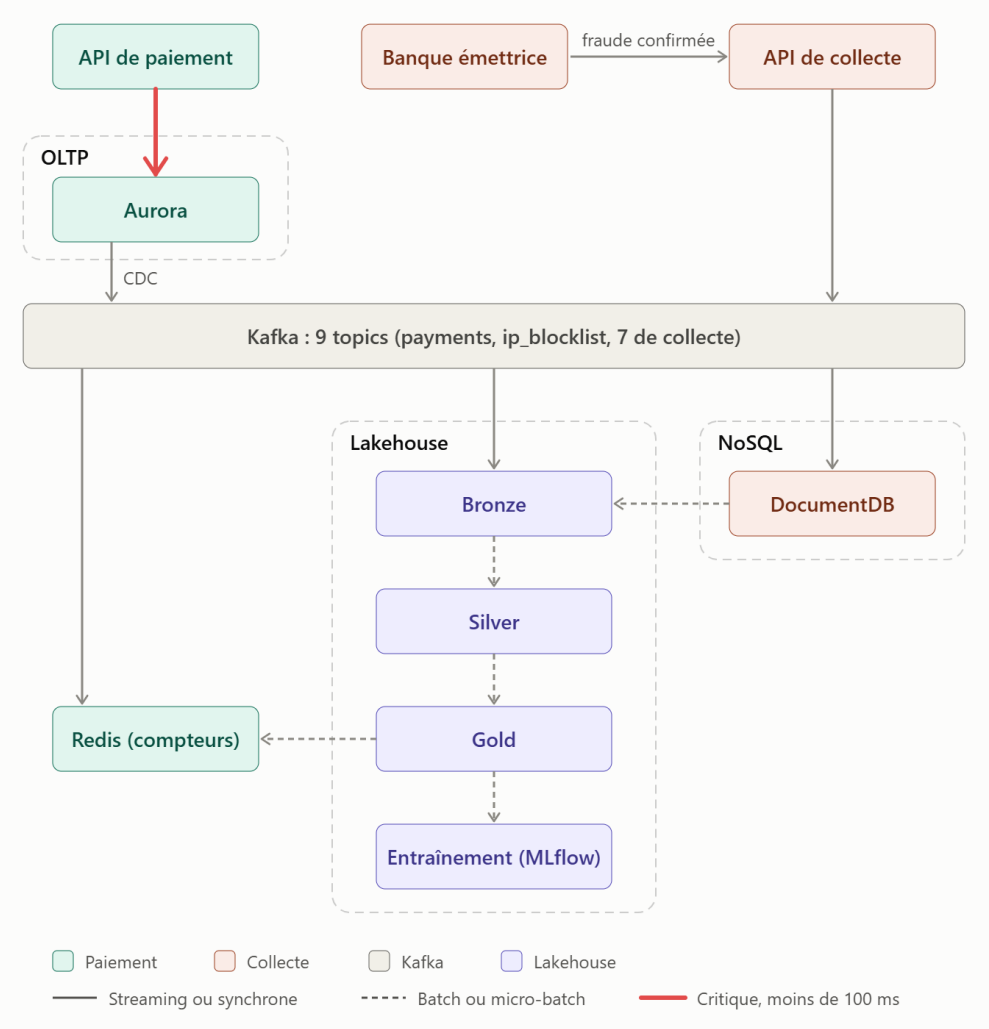
Le schéma ci-dessous montre comment les données circulent entre les trois systèmes. Il se lit de haut en bas.
Deux flux entrent dans le pipeline :

- Le **flux critique** est celui du paiement : l'API de paiement écrit le paiement, son évaluation de fraude et l'événement (outbox) dans Aurora, en une seule transaction et en moins de 100 ms. Debezium lit ensuite l'outbox et publie les événements dans Kafka, sans ralentir le paiement.
- Le **flux secondaire** est celui de la collecte des données hors paiement : l'API de collecte publie directement dans Kafka les logs, les clics, les sessions et les avis, ainsi que les fraudes confirmées renvoyées par la banque émettrice, sans passer par Aurora.

Kafka distribue ensuite les messages. Les événements de paiement alimentent les compteurs Redis en quelques secondes et arrivent dans la couche Bronze du lakehouse. Les données de collecte sont stockées dans DocumentDB, puis exportées chaque nuit vers Bronze.

Dans le lakehouse, les données sont transformées par lots : Bronze (brutes), Silver (nettoyées et sans doublon), puis Gold (prêtes pour la BI). Gold alimente les features froides de Redis et le réentraînement du modèle dans MLflow.

Les fraudes confirmées par la banque émettrice arrivent par l'API de collecte, rejoignent Bronze puis Gold, et servent de cible d'apprentissage au réentraînement (voir 3.4).



3 Vue détaillée par flux

Le document suit les liens entre les 3 systèmes : de l'OLTP vers le lakehouse, du NoSQL vers le lakehouse, puis du lakehouse (et de Kafka) vers Redis et MLflow. La dernière partie décrit la boucle de feedback : les fraudes confirmées par la banque émettrice, qui reviennent dans le lakehouse pour le réentraînement.

3.1 De l'OLTP vers le lakehouse

Les paiements suivent le flux critique : Aurora, Kafka, puis Bronze, Silver et Gold. Les tables de référence d'Aurora rejoignent Bronze par un export quotidien.

3.1.1 Écriture du paiement

L'API de paiement écrit dans Aurora PostgreSQL, en mode synchrone et en moins de 100 ms. Elle écrit trois lignes dans la même transaction : le paiement, son évaluation de fraude et l'événement à publier dans l'outbox. Elles existent toutes ou aucune n'existe. Un remboursement est une nouvelle ligne transaction, liée au paiement d'origine.

La table idempotency_key évite qu'une requête rejouée par le marchand crée un second paiement. Si Kafka est indisponible, les paiements continuent et les événements attendent dans l'outbox.

3.1.2 Publication dans Kafka

Debezium lit les nouvelles lignes de l'outbox et les publie dans Kafka, en quelques secondes et sans ralentir le paiement. Il route chaque événement selon aggregate_type vers l'un des deux topics issus de l'outbox.
Le topic payments contient les paiements (payment) et les remboursements (refund). Un paiement et son remboursement sont publiés avec l'identifiant du paiement d'origine comme clé, donc dans la même partition : ils sont traités dans l'ordre.

Ce topic est lu par Bronze et par Redis. Le topic ip_blocklist contient les ajouts et retraits de plages IP bloquées, et il est lu uniquement par Redis.

La livraison est « au moins une fois » : un message peut arriver en double, d'où des consommateurs idempotents. Chaque consommateur lit les topics à son rythme, sans gêner les autres. Si l'un tombe en panne, il reprend là où il s'est arrêté, et peut relire des messages anciens tant que Kafka les conserve.

Un message impossible à traiter est mis de côté dans une file d'erreurs (DLQ) pour être analysé.

3.1.3 Arrivée dans Bronze

Bronze (Delta Lake sur S3) conserve les messages tels que reçus : ajout seul, aucun rejet, aucune correction. C'est la source de rejeu : si Silver ou Gold sont faux, on les recalcule depuis Bronze. Il reçoit les données d'Aurora par deux chemins.

Le topic payments est écrit en streaming ou en micro-batch, avec une fraîcheur de quelques minutes. Chaque ligne garde le message brut et ses métadonnées Kafka (topic, partition, offset, date d'ingestion).

Un lot Workflows exporte aussi chaque jour les tables de référence (merchant, customer, product, country, currency et analytic_category) depuis un réplica de lecture d'Aurora (outbox), pour ne pas charger l'instance d'écriture. Cet export est chargé avant la transformation Silver puis Gold : sans lui, un paiement pourrait arriver sans son marchand ou son produit en BI.

Les données personnelles de la table customer relèvent du document sécurité et conformité.

3.1.4 Transformation : Bronze, Silver, Gold

Les données de Bronze sont transformées par lots en deux étapes : Silver (propre, sans doublon), puis Gold (prêt pour l'usage), sur Delta Lake (S3).

Un lot Workflows traite Bronze vers Silver en micro-batch, avec une fraîcheur de quelques minutes. Les messages bruts sont lus et convertis en colonnes typées, puis validés (identifiants présents, montant positif, devise connue…). Les lignes rejetées vont en quarantaine avec leur motif, pour correction à la source. Aucune ligne n'est perdue.

L'insertion se fait par MERGE sur l'identifiant de l'événement : rejouer le traitement donne le même résultat, ce qui compense la livraison « au moins une fois » de Kafka.

Un lot Workflows alimente ensuite Gold, chaque heure ou chaque jour. Les dimensions sont chargées avant les faits, sinon un paiement pourrait arriver sans son marchand ou son produit. Les dimensions couvrent le marchand, le client, le produit, le pays, la catégorie et la date. Les faits comptent une ligne par transaction (paiement ou remboursement) et une ligne par produit d'un paiement. Des tables dérivées complètent l'ensemble : des agrégats pour les tableaux de bord, et les features froides destinées à Redis.

3.2 Du NoSQL vers le lakehouse

Les données de collecte suivent le flux secondaire : API de collecte, Kafka, DocumentDB, puis un export de nuit vers Bronze. Elles ne passent jamais par Aurora.

3.2.1 Collecte et topics

L'API de collecte reçoit les logs, clics, sessions et avis des sites et applications, ainsi que les fraudes confirmées renvoyées par la banque émettrice, et les publie directement dans Kafka, en quelques secondes.

Il y a un topic par type de données : sessions, click_events, logs, customer_feedback, ml_features et inbound_payloads (une par collection), et fraud_labels (voir 3.4).
Les six premiers sont lus par DocumentDB, qui les stocke. Trois d'entre eux (sessions, click_events et ml_features) sont aussi lus par Redis, qui en tire le compteur C8.

Le topic fraud_labels n'est pas stocké dans DocumentDB : il est lu uniquement par Bronze.

3.2.2 Stockage dans DocumentDB

DocumentDB conserve les données de collecte sous forme de documents JSON, en quelques secondes. Un consommateur lit les six topics et écrit une collection par topic, du même nom.

L'écriture se fait par identifiant de document (upsert), donc un message livré en double ne crée pas de doublon. Chaque document porte un expires_at et il est supprimé automatiquement après 90 jours par exemple (TTL).

Pour inbound_payloads, le message est stocké tel quel et les fichiers binaires sont déposés dans S3 : le document garde seulement le chemin du fichier.

Les messages impossibles à écrire partent dans une DLQ.

3.2.3 Export de nuit vers Bronze

Un lot Workflows exporte chaque nuit les six collections (sessions, click_events, logs, customer_feedback, ml_features et inbound_payloads) vers Bronze, avec une fraîcheur de J+1. Seuls les documents nouveaux depuis le dernier export sont lus. Les données suivent ensuite la même transformation que les paiements.

3.3 Alimentation de Redis et sorties de Gold

3.3.1 Redis

Redis (ElastiCache) contient les compteurs de fraude, à haute disponibilité. Il ne contient aucune donnée de référence et peut être reconstruit à partir des sources.

Le flux chaud est alimenté en streaming, en quelques secondes. Un premier consommateur lit payments et ip_blocklist : il met à jour les compteurs C1 à C7 et la liste de blocage. Un remboursement approuvé ajuste C4 (montant cumulé), les autres compteurs ne le comptent pas. Un second consommateur lit sessions, click_events et ml_features et met à jour C8 (profil de session avant paiement). Chaque compteur a un TTL et ses clés sont pseudonymisées.

Le flux froid est alimenté par lot, chaque jour : un lot Workflows lit les features froides dans Gold et les publie dans Redis. Les features issues de DocumentDB (ml_features) suivent le chemin DocumentDB, Bronze, Silver, Gold, puis Redis. Le même lot publie les recommandations de personnalisation, calculées dans Databricks à partir de Gold.

3.3.2 Autres sorties de Gold

Gold alimente aussi la BI, MLflow pour l'entrainement, et les lots Databricks de personnalisation et d'analyses prédictives (livrable 5). Les recommandations sont publiées dans Redis, les prévisions à la BI.

3.4 Boucle de feedback : étiquettes de fraude

Le modèle apprend des fraudes réellement confirmées. Cette information vient de la banque émettrice, plusieurs jours après le paiement. Elle passe par l'API de collecte (flux secondaire) et non par Aurora, car elle sert uniquement au réentraînement et n'a pas besoin d'être en temps réel.

3.4.1 Collecte

La banque envoie la fraude confirmée à l'API de collecte, qui la publie dans un topic Kafka dédié, fraud_labels, avec l'identifiant de la transaction pour clé. L'API répond après l'accusé de réception de Kafka. L'idempotence repose sur cet identifiant : une étiquette renvoyée deux fois ne crée pas de doublon, et la banque peut renvoyer sans risque.

3.4.2 Arrivée dans Bronze

Bronze lit fraud_labels en micro-batch, sans passer par DocumentDB : c'est une donnée structurée, liée à une transaction.

3.4.3 Transformation

En Silver, l'étiquette est validée : la transaction doit exister, sinon elle va en quarantaine. L'insertion se fait par MERGE sur l'identifiant de la transaction, la dernière valeur remplaçant la précédente si la banque corrige. En Gold, l'étiquette est jointe à la transaction. Rien n'est réécrit dans Aurora.

3.4.4 Réentraînement

MLflow lit Gold, où chaque transaction porte sa vraie valeur de fraude, et s'en sert comme cible d'apprentissage. Le cycle se referme : le modèle produit des scores, les banques confirment les fraudes, et ces confirmations entraînent la version suivante.

4 Orchestration

L'orchestration ordonne les traitements du pipeline, avec leurs dépendances, leurs relances et leurs alertes. Elle est assurée par Databricks Workflows pour les traitements par lots. Les flux continus tournent en permanence et ne sont pas planifiés.

Job	Type	Fréquence
Debezium : outbox vers Kafka	Continu	En permanence
API de collecte vers Kafka	Continu	En permanence
Kafka vers DocumentDB	Continu	En permanence
Kafka vers Redis (compteurs)	Continu	En permanence
Kafka vers Bronze (paiements)	Continu	En permanence
Export de DocumentDB vers Bronze	Planifié	Chaque nuit
Kafka vers Bronze (étiquettes de fraude)	Continu	En permanence
Export des tables de référence d'Aurora vers Bronze	Planifié	Chaque nuit
Bronze vers Silver	Planifié	Toutes les quelques minutes
Silver vers Gold	Planifié	Chaque heure ou chaque jour
Features froides vers Redis	Planifié	Chaque jour
Réentraînement (MLflow)	Planifié	Chaque semaine
Recommandations de personnalisation (Gold vers Redis)	Planifié	Chaque jour
Analyses prédictives (Databricks)	Planifié	Chaque jour

Dépendances

- Les deux exports de nuit sont indépendants et s'exécutent en parallèle.
- Dans Gold, les dimensions sont chargées avant les faits.
- Les features froides, les lots de personnalisation et de prévision, et le réentraînement démarrent une fois Gold à jour.

Relances et alertes

- Chaque job planifié est relancé automatiquement en cas d'échec. Comme les traitements sont idempotents, une relance ne crée pas de doublon.
- Si l'échec persiste, les étapes suivantes ne démarrent pas et une alerte part de CloudWatch.

5 Monitoring et reprise

5.1 Monitoring

CloudWatch collecte les métriques du pipeline et alerte l'équipe d'exploitation dans 5 cas : le retard des consommateurs Kafka, la fraîcheur des tables (un job qui n'a pas tourné à l'heure prévue), le taux de rejet en quarantaine et en DLQ, l'échec d'un job planifié et le délai entre un paiement et l'arrivée de son étiquette de fraude car sans retour de la banque, le modèle ne s'améliore plus.

5.2 Reprise sur incident

Si Kafka est indisponible, les paiements continuent et les événements attendent dans l'outbox, puis sont publiés au retour de Kafka. Un consommateur en panne reprend là où il s'est arrêté, ou rejoue les messages tant que la rétention de Kafka le permet.

Si Silver ou Gold sont erronés, on les recalcule depuis Bronze, qui conserve les messages tels que reçus. En cas d'erreur de traitement sur une table, Delta permet de revenir à une version précédente (RESTORE).

Redis, enfin, se reconstruit à partir des sources, car il ne contient aucune donnée de référence.
La conformité RGPD (effacement dans Bronze, Silver et Gold) est traitée dans le document sécurité et conformité.